# CSC3060 Project 3
## Code Scheduling

Ming Wen
mingwen@link.cuhk.edu.cn
CSC3060 Teaching Team

Spring 2026
Version 1.0.0

## 1 Assignment Overview

From the lecture, we learned data forwarding as a hardware solution to hide latency and reduce pipeline stalls. And from software there are also solutions for this issue. One representative approach is Code Scheduling, which reorders instructions at compile time using list scheduling. By analyzing data dependencies and hardware resource constraints, the scheduler rearranges independent instructions to overlap with long-latency operations. This increases instruction-level parallelism, reduces pipeline bubbles, and improves overall execution efficiency without changing program semantics.

In this Assignment, we provide a minimal implementation of LLVM's code scheduling module, and you will build most of the components needed to schedule RISC-V Assembly efficiently!

**Deadline: 2026-03-26 23:59 China Standard Time**

### What you will implement

1. Basic Abstraction for scheduling process(e.g. Instruction, DAG, DAGNode).
2. A Directed Acyclic Graph(DAG) Builder to build dependency between instructions.
3. A List Scheduler to form the final scheduling result.

### What you can use

The project is a minimal implementation of List Scheduling and only require CTest as the only Third Party Libary for unit testing, which means you can finish all the task using just C++.

**Statement on AI tools** Prompting LLMs such as ChatGPT is permitted for low-level programming questions or high-level conceptual questions about code schedluing, but using it directly to solve the problem is prohibited. **If you use LLMs, include the conversation LINK in the end of your report.**

We Strongly encourage you to disable AI autocomplete(e.g. Github CoPiolt) in your IDE when completing tasks(though non-AI autocomplete, e.g., autocompleting function names is totally fine). We have found that AI autocomplete makes it much harder to engage deeply with the content.

**What the code looks like** All the assignment code are available on GitHub at:

`https://github.com/pandaman176/CSC3060-Project3.git`

Please `git clone` the repository. If there are any updates, we will notify you so you can `git pull` to get the latest.

1. src/*: This is where source code exists
2. test/*: Unit Testing files (DO NOT MODIFY)
3. submmision.py: python script to help you zip all stuff.
4. CMakeLists.txt: for build and compilation

**How to submit** You will submit the following files to BlackBoard(BB):

- studentId_report.pdf: Answer all disscusion problem and share you thoughts
- submission.zip: Contain all the code.

We **strongly recommend** you to use python script to generate `submission.zip` By running:

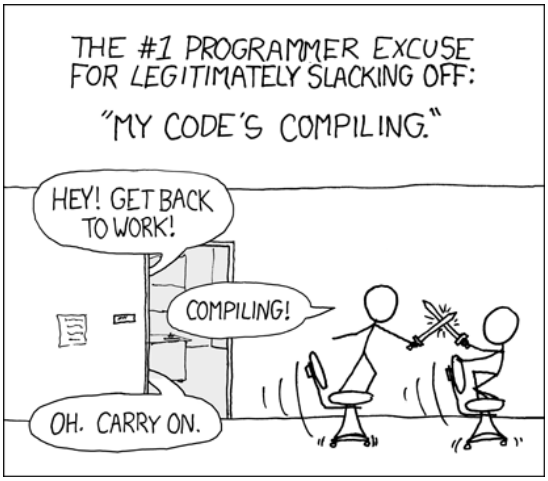
`python3 submission.py`

**How to Compile**

```
# CMake 3.14 or Higher
# C++ Compiler

# macos/linux/wsl(recommend)
mkdir build && cd build
cmake ..
make -j4

# windows
mkdir build && cd build
cmake .. -G "Visual Studio 16 2019"
cmake --build . --config Release
Release\scheduler.exe
```




## 2 Abstraction and Latency Model

### 2.1 `Instruction` Abstarction

The code scheduling process contains four parts:

1. Parsing Assembly code blocks
2. Build Dependency Graph(A DAG!)
3. Calculate Priority
4. Forming a Final Scheduling

The `runScheduler()` function in `main.cpp` will follows the above four steps plus a analysis stage to visualize the scheduling result but now it is not implemented. To concentrate on the core logic of scheduling, we already simplyfied the instruction model and implemented the parsing stage for you, which means you can directly start build dependency graph on it! However, it is worthwhile to spend some time first becoming familiar with how this project abstracts `instructions`.

```
class Instruction {
public:
    int index;
    std::string raw_text;
    std::string opcode;
    std::string dest_reg;
    std::vector<std::string> src_regs;
    int latency;

    // Constructors
    Instruction();
    Instruction(int idx, const std::string& raw, const std::string& op);

    bool hasDestReg() const;
    bool usesReg(const std::string& reg) const;
    bool definesReg(const std::string& reg) const;

    void print(std::ostream& os = std::cout) const;
};
```

The Instruction class provides a simplified abstraction of a RISC-V instruction aimed at dependency analysis for instruction scheduling. It captures only the essential information needed by the scheduler: opcode, source and destination registers, latency, and original program order, while deliberately omitting details such as immediates and labels.

**Problem ( Instruction abstraction [5 point] ):**

To warm up and test you understanding of the basic abstractions, you need implement two function to help check the instruction use which registers as source and defines which register as destination

**Deliverable:** Implement `useReg()` and `definesReg()` (src/instruction.cpp)

To test your function on our provided test, first `cd build` and `make -j4` then run `ctest -R InstructionTest` You Implementation should be able to pass all tests.

**Ctest Tip**

Instead of directly excute test files, Ctest provider Command Line Tool `ctest`: Running ctest executes all registered tests, while `ctest -j` enables parallel execution to speed up testing. For debugging, `ctest -V` (or ctest –verbose) shows detailed test output. Users can also run specific tests with `ctest -R <pattern>` and rerun only failed tests using `ctest -rerun-failed`.

### 2.2 Static Latency Model

Code scheduling relies on a latency model to decide how far apart dependent instructions must be placed to avoid pipeline stalls. Without latency information, the scheduler cannot distinguish between short, single-cycle operations and long-latency instructions such as loads or divisions, leading to suboptimal schedules. LLVM uses a target-specific latency model derived from the processor's scheduling description, where instruction latencies and resource usage are modeled

| Instruction Category | Example Opcodes | Latency (cycles) |
|---|---|---|
| Arithmetic / Logic | add, sub, and, or, xor | 1 |
| Shift / Compare | sll, srl, sra, slt | 1 |
| Immediate / Upper Immediate | addi, lui, auipc | 1 |
| Multiplication | mul, mulh, mulw | 3 |
| Division / Remainder | div, divu, rem | 10 |
| Load (cache hit assumed) | lb, lh, lw, ld | 5 |
| Store | sb, sh, sw, sd | 1 |
| Branch / Jump | beq, bne, jal, jalr | 1 |
| Pseudo-instructions | mv, li, la, nop | 1 |
| Unknown instruction | – | 1 |

Table 1: Simplified static instruction latency model

per microarchitecture and integrated into its machine scheduler for more accurate and hardware-aware scheduling decisions. In this project, we adopt a simplified static latency model that assigns a fixed execution latency to each opcode via a lookup table, assuming ideal conditions (e.g., cache hits for loads).

**Problem ( Static latency model [2+2+5 point] ):**

(a) We know both load and store are memory instructions, why in static latency model we can assume load have 5 clcyle latency while store only take 1 cycle latency?
**Deliverable:** A one to two sentence responce

(b) Why a static latency model makes sense even though real instructions do not always take a fixed number of cycles?
**Deliverable:** A one to two sentence responce

(c) There are some code missing in the `getInstructionLatency()` function, please fix it
**Deliverable:** Fix `getInstructionLatency()` (src/instruction.cpp)

To test your function on our provided test, first `cd build` and `make -j4` then run `ctest -R LatencyTest` You Implementation should be able to pass all tests.

## 3 Dependency Graph

One more thing to address before scheduling is dependencies. We definitely hope after scheduling the code still generate same result. To do that we need Dependency Graph. Imagine instructions as task in a factory. Each task uses some materials(register, memory) and produces some materials. A dependency graph is a graph that says:

"This instruction must happen before that instruction."

Formally, each node represents an instruction and each edge represent different types of dependencies(ordering constraints), and we will go through them one by one and implement dependency checker finally.

### 3.1 Flow Dependency: Read After Write(RAW)

As the name says, this dependency means that an instruction reads a value that was written by an earlier instruction, below is a simple example:

- I1: mul r1, r2, r3 (latency = 3)
- I2: add r4, r1, r5

Since I2 is waiting for the value in r1, I2 cannot execute until value in r1 is actually produced. RAW is the only dependency that carries true timing information. $start(I2) \geq start(I1) + latency(I1)$

### 3.2 Anti Dependency: Write After Read(WAR)

WAR meaning: "Don't overwrite a register until earlier reads are done."

- I1: add r4, r1, r2 ; reads r1
- I2: sub r1, r3, r5 ; writes r1

I1 must start before I2, once I1 has issued, it alredy read r1, we do not need to wait for I1 to finish. $start(I2) \geq start(I1)$

### 3.3 Output Dependency: Write After Write (WAW)

It means two instruction write to the same destination, and the ordering determines the final valaue of register.

- I1: add r1, r2, r3
- I2: sub r1, r4, r5

So again, this is about ordering, not data readiness. $start(I2) \geq start(I1)$

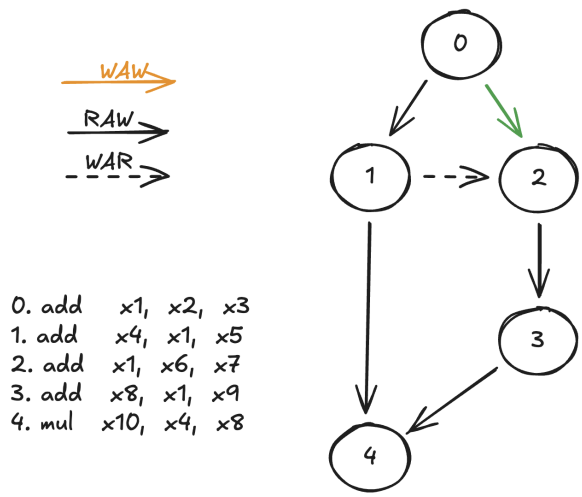



Figure 1: Dependency Graph

Figure. 1 shows the complete graph for a code piece, you can use it to test whether you understand the concept. The remaining question is how to build it: We will demonstrate how to find all RAW dependencies in one pass 1, and you need to figure out how to do WAR and WAW to finish the rest by yourself:

**Algorithm 1** Detect Read-After-Write (RAW) Dependencies

**Require:** Instruction sequence $I = \langle I_0, I_1, \ldots, I_{n-1} \rangle$
**Require:** DAG node list $N$
**Ensure:** RAW dependency edges added to $N$

```
Initialize empty map LastWriter
for i ← 0 to n − 1 do
    inst ← I_i
    for all r ∈ inst.src_regs do
        if r ∈ LastWriter then
            j ← LastWriter[r]
            lat ← I_j.latency
            Add edge (j → i) of type RAW on register r
        end if
    end for
    if inst writes a destination register d then
        LastWriter[d] ← i
    end if
end for
```

**Problem ( Dependency Detection [2+2+10+10+[5] point] ):**

(a) Why there is no RAR Dependency?
**Deliverable:** A one to two sentence responce

(b) Why we say Dependency Graph should be a DAG, what if not?
**Deliverable:** A one to two sentence responce

(c) Implement WAW and WAR detection
**Deliverable:** `detectWAR(), detectWAW()` (src/DAGBuilder.cpp)

(d) **Bonus**: If you read `DAGBuilder:build`, you will find out detectWAW/WAR/RAW can be done together, can you do it in one pass?
**Deliverable:** one pass function to build DAG

To test your function on our provided test, first `cd build` and `make -j4` then run

```
ctest -R "DAGBuilder|Integration"
```

You Implementation should be able to pass all tests.

For the Bonus Part, you can omit previous `detectXXX` functions we provided and rewrite the `DAGBuilder:build` by yourself, make sure it still pass all test case above.

**Simplify Assumption We made and Why (Optional)**

The real world is always complex and buggy. To make the project clean and pedagogical, we implicitly assuming a very simplified execution model:

- All dependencies are register dependencies
- No memory aliasing
- Single-threaded (single process) program
- No control-flow / exception / side effects
- Instructions are atomic w.r.t. memory

Under these assumptions, Registers are the **only** shared state , so all correctness constraints reduce to who reads which register and who write which register, and those constraints are exactly: RAW, WAR, WAW. Let's use some examples to show how everything blows up without these assumption:

What if there is memory aliasing, :

- sd x10, 0(x5) # store x10 -> mem[x5+0]
- ld x11, 32(x6) # load mem[x6+32] -> x11

if x5+0 is same as x6+32, i.e. they point to same place in memory, those two instruction form a RAW in memory level. But if you only check register, they are independent.

Let's see a more tricky case: in multi-thread program, usually we use lock to protect share variable:

- I1: amoswap.w.aq t0, t0, (x1) //acquire lock stored in mem[x1]
- I2: sd x1, 0(x10) // x = 1 (modify share variable in mem[x10])
- I3: sd zero, 0(x1) // release lock in mem[x1]

However, from scheduler view, I2, I3 is independent, and it is allowed to reschedule as I1 -> I3 -> I2, but it is a disaster for parallel program since shared variable is modified after lock released.

If you are interest in the world of parallel programming, I strongly recommend that you take CSC4005 later, which would be an unforgettable experience!

## 4 Priority and Scheduling

Now that we have built the dependency graph, we can proceed to the core scheduling algorithm. The goal is to reorder instructions to minimize total execution cycles while respecting all dependencies. The key insight is: **instructions on the critical path should be scheduled as early as possible**.

### 4.1 List Scheduling Algorithm

List scheduling is one of the most fundamental instruction scheduling algorithms used in production compilers:

**Algorithm 2** List Scheduling with Inflight Queue

**Require:** Instruction sequence $I$, DAG nodes $N$ with computed priorities
**Ensure:** Scheduled order $S$, total cycles

```
cycle ← 0; S ← []
ready ← {n ∈ N : n.unscheduled_predecessors = 0}
inflight ← {}                                          ▷ set of (node, finish_cycle) pairs
while |S| < |N| do
    if ready ≠ ∅ then                                  ▷ issue one instruction (issue width = 1)
        best ← arg max_{n∈ready} n.priority; remove best from ready
        Append best to S; best.scheduled ← true
        Add (best, cycle + latency(best)) to inflight
        for all WAR/WAW successor s of best do         ▷ order-only: release on issue
            s.unscheduled_predecessors −= 1
            if s.unscheduled_predecessors = 0 then add s to ready
            end if
        end for
    end if
    cycle ← cycle + 1
    for all (op, t) ∈ inflight where t = cycle do      ▷ retire finished ops
        Remove (op, t) from inflight
        for all RAW successor s of op do               ▷ data ready: release on finish
            s.unscheduled_predecessors −= 1
            if s.unscheduled_predecessors = 0 then add s to ready
            end if
        end for
    end for
end while
return S, cycle
```

**Key Insight: When to Release Successors**

The inflight queue captures the distinction between two types of constraints:

- **RAW (true dependency):** the successor needs the *result*, so it can only be released when the predecessor *finishes* ($cycle = issue_cycle + latency$).
- **WAR/WAW (anti-dependency):** the constraint is only about *ordering*, not data readiness. Once the predecessor is issued, the register has been read (WAR) or the write order is fixed (WAW), so the successor can be released immediately.

### 4.2 Priority: Critical Path Length

The priority of a node determines which instruction to schedule first when multiple instructions are ready. We use **Critical Path Length** (CPL) as the priority metric:

$$Priority = longest\ path\ from\ current\ node\ to\ any\ exit\ node$$

Mathematically, for node $n$:

$$priority(n) = \begin{cases} latency(n) & \text{if } n \text{ has no successors} \\ \max \begin{cases} latency(n) + \max\limits_{s \in RAW_succ(n)} priority(s) \\ \max\limits_{s \in WAR/WAW_succ(n)} priority(s) \end{cases} & \text{otherwise} \end{cases} \tag{1}$$

where $RAW_succ(n)$ are successors connected by RAW edges, and $WAR/WAW_succ(n)$ are successors connected by WAR or WAW edges.

**Key insight:** RAW dependencies carry true timing constraints (the successor must wait for the predecessor to complete), so we add the current node's latency. WAR and WAW dependencies only enforce ordering (no timing delay), so we don't add the current node's latency for these edges.

**Why CPL?**

Instructions with longer critical paths have more dependent work waiting for them. Scheduling them earlier allows their dependent instructions to start sooner, reducing overall execution time.

### 4.3 A Complete Example

Consider the following code block with 5 instructions:

```
I0: lw   x1, 0(x5)      ; latency=5, writes x1
I1: lw   x2, 4(x5)      ; latency=5, writes x2
I2: add  x3, x6, x7     ; latency=1, independent
I3: mul  x4, x1, x2     ; latency=3, reads x1, x2
I4: sw   x4, 8(x5)      ; latency=1, reads x4
```

**Step 1: Build DAG**

$$\begin{aligned} &I0 \to I3 \quad (\text{RAW } x1,\ \ell = 5) \\ &I1 \to I3 \quad (\text{RAW } x2,\ \ell = 5) \\ &I3 \to I4 \quad (\text{RAW } x4,\ \ell = 3) \\ &I2 \text{ independent} \end{aligned}$$

**Step 2: Priority Computation**

$$\begin{aligned} &priority(I4) = 1, \quad priority(I2) = 1 \\ &priority(I3) = 3 + 1 = 4 \\ &priority(I0) = 5 + 4 = 9, \quad priority(I1) = 5 + 4 = 9 \end{aligned}$$

**Step 3: List Scheduling**

Each row shows: *cycle — ready set — issued instruction — inflight (node@finish).* RAW successors are released only when the predecessor finishes; WAR/WAW successors are released immediately on issue.

$C0: \ \{I0, I1, I2\} \rightarrow \text{issue } I0, \quad inflight = \{I0@5\}$

$C1: \ \{I1, I2\} \rightarrow \text{issue } I1, \quad inflight = \{I0@5, \ I1@6\}$

$C2: \ \{I2\} \rightarrow \text{issue } I2, \quad inflight = \{I0@5, \ I1@6, \ I2@3\}$

$C3: \ I2 \text{ retires (no RAW succ)}, \quad ready = \{\}$

$C4: \ \text{idle } (I0, I1 \text{ still in-flight})$

$C5: \ I0 \text{ retires} \Rightarrow I3.pred: 2 \rightarrow 1, \quad ready = \{\}$

$C6: \ I1 \text{ retires} \Rightarrow I3.pred: 1 \rightarrow 0 \Rightarrow ready = \{I3\} \rightarrow \text{issue } I3, \quad inflight = \{I3@9\}$

$C7\text{–}C8: \ \text{idle}$

$C9: \ I3 \text{ retires} \Rightarrow I4.pred: 1 \rightarrow 0 \Rightarrow ready = \{I4\} \rightarrow \text{issue } I4, \quad inflight = \{I4@10\}$

**Final Schedule:** $\langle I0, \ I1, \ I2, \ I3, \ I4 \rangle, \quad total_cycles = 10$

*Note:* The issue order happens to be the same as the original program order in this example. The total of 10 cycles is determined by the critical path: $\max(latency(I0), \ latency(I1)) + latency(I3) + latency(I4) = \max(5, 6) + 3 + 1 = 10$, which is the absolute lower bound no scheduler can beat. What list scheduling achieves here is filling the idle slot at C2 with the independent instruction I2, hiding two cycles of latency that would otherwise be wasted stalls.

**Problem ( Priority Calculation [4+10+10 point] ):**

(a) Why WAR and WAW dependencies only enforce ordering (i.e. no timing delay)
**Deliverable:** A one to two sentence responce

(b) Implement the critical path length calculation for scheduling priority.
**Deliverable:** Implement `computeCriticalPath()` (src/Scheduler.cpp)

(c) Implement List Scheduling process with an inflight queue. Your implementation must correctly distinguish between RAW and WAR/WAW edges: WAR/WAW successors are released when the predecessor is *issued*; RAW successors are released only when the predecessor *finishes* (i.e., at $issue_cycle + latency$).
**Deliverable:** Implement `schedule()` (src/Scheduler.cpp)

To test:

```
cd build && make -j4
ctest -R
"SchedulerTest.ComputePriorities|SchedulerTest.Schedule_"
```

### 4.4 Tie-Breaking Policies

When multiple ready nodes have the same priority, we need a **tie-breaking policy** to decide which one to schedule first. Different policies can lead to different schedules with potentially different performance characteristics:

1. **SMALLER_INDEX**: Select the node with smaller original index. This preserves program order when priorities are equal.

2. **MOST_CHILD**: Select the node with the most successors (children). Scheduling nodes with more dependents earlier can unblock more instructions sooner.

3. **LPT (Longest Processing Time)**: Select the node with the longest latency. This prioritizes long-latency instructions to start early, allowing other instructions follows it to hide its latency.

**Problem ( Tie-Breaking Policies [2+2+10 point] ):**

(a) Why might MOST_CHILD be a good tie-breaking strategy? Give a brief example scenario.
**Deliverable:** A one to two sentence response

(b) In what scenario would LPT outperform SMALLER_INDEX?
**Deliverable:** A one to two sentence response

(c) Implement all three tie-breaking policies in `selectBestNode()`, if two candidates still tie on MOST_CHILD or LPT, use SMALLER_INDEX to break the tie.
**Deliverable:** Implement tie-breaking in `selectBestNode()` (src/Scheduler.cpp)

To test: `cd build && make -j4 && ctest -R TieBreaking`

### 4.5 Complexity Analysis

Let $n$ be the number of instructions and $e$ be the number of dependency edges.

**Problem ( Complexity Analysis [2+2+[5] point] ):**

(a) What is the time complexity of the naive list scheduling implementation (the one that scans all nodes to find ready nodes each iteration)?
**Deliverable:** Express in Big-O notation with brief explanation

(b) What is the time complexity of priority calculation using the recursive memoized approach?
**Deliverable:** Express in Big-O notation with brief explanation

(c) **Bonus**: The current implementation rebuilds the ready list by scanning all nodes each iteration. How can we use a priority queue (max-heap for ready nodes, min-heap for inflight retirement) to optimize this? What would be the new time complexity? Can you give the code?
**Deliverable:** Brief description and complexity analysis; implement `scheduleFast()` (src/Scheduler.cpp). Your optimized version must also use an inflight queue with the same RAW/WAR/WAW release semantics as `schedule()`.

To test: `cd build && make -j4 && ctest -R SchedulerPerformance`

**Hints: Priority Queue in C++**

C++ provides `std::priority_queue` in `<queue>`. By default it's a max-heap. To customize comparison:

```
auto cmp = [&](int a, int b) {
    return nodes[a].priority < nodes[b].priority;
};
std::priority_queue<int, std::vector<int>,
                    decltype(cmp)> pq(cmp);
```

Remember: the comparison returns true if the first argument should have **lower** priority than the second.

## 5 End to End Integration

Congratulations on getting to this point. You're almost done! Now, you already finished all components and it's time to put them together. We prepared several code blocks in `main.cpp` And you need to schedule these blocks by following steps: Parse -> Build DAG -> Schedule

**Problem ( Run Scheduler [10+10 point] ):**

(a) Schdule Code blocks in `main.cpp`, the following are some functions you should use:

- Parser::parse()
- DAGBuilder::build()
- Scheduler::schedule()
- Scheduler::printSchedule()
- Scheduler::printComparison()

**Deliverable:** Implement `runScheduler()` and analysis schedule results (src/main.cpp)

There are no test for `runSchedule`, instead you should analysis the result of example codes, does the scheduling improve performance? Why or Why not? Write your finding into report.

## 6 Grading Rubric

### Point Distribution

| Problem | Deliverable | Points |
|---|---|---|
| *Code Problems — 70 pts* | | |
| Instruction Abstraction | `usesReg()`, `definesReg()` | 5 |
| Static Latency Model | `getInstructionLatency()` | 5 |
| Dependency Detection | `detectWAR()`, `detectWAW()` | 20 |
| Priority & Scheduling | `schedule()`, `computeCriticalPath()` | 20 |
| Tie-Breaking Policies | `selectBestNode()` | 10 |
| End-to-End Integration | `runScheduler()` | 10 |
| *Written Problems — 30 pts* | | |
| Static Latency Model | Questions (a)(b) | 4 |
| Dependency Detection | Questions (a)(b) | 4 |
| Priority Calculation | Question (a) | 4 |
| Tie-Breaking Policies | Questions (a)(b) | 4 |
| Complexity Analysis | Questions (a)(b) | 4 |
| End-to-End Integration | Schedule analysis & findings | 10 |
| *Bonus — up to +10 pts* | | |
| Dependency Detection | One-pass DAG builder | 5 |
| Complexity Analysis | `scheduleFast()` with priority queue | 5 |
| | | **Total** **100 + 10** |

Table 2: Grading rubric point distribution

### Grading Detailes

All code deliverables that have `ctest` tests are graded by the following threshold:

| Test pass rate | Score awarded |
|---|---|
| 100% | Full marks |
| $> 80\%$ | Half marks |
| $\leq 80\%$ | 0 |

The integration task (`runScheduler`) has no automated tests and is graded manually by the teaching team.

All written deliverables should be concise, to the point and written in **english**. We prefer your own thinking rather than copied GPT answers. As long as your answer is in the right direction, you should receive full marks, so don't worry too much on that.

### 6.1 Bonus Policy

If you have done the Bonus Part, please mention that in your report, notice that the bonus will count as part of the participation component of the final grade.

### 6.2 Late Policy

A penalty of 10 percent of the actual score will be applied for each day after the deadline (**2026-03-26 23:59 China Standard Time**). Submissions more than three days late will receive a score of **zero**.

Please start this project early. If you encounter any unexpected issues, email the Teaching Team to request an extension with a valid excuse and proof.

### 6.3 Academic Integrity

Understand your code. You must be able to explain every line of code that you write.

- Proper attribution. If you use external resources, such as algorithms from textbooks, websites, or friends, or LLMs, cite them in your code with comments, and explicitly mention them in your report.
- If you use LLMs, include the conversation Link in your report.

### 6.4 Interview Policy

- If you claim that you do not use any LLM tools, we will definitely interview you.
- If your report does not match your code, we will interview you.
- We may interview you about other suspicious things.